Modul 122, Dokumentation LB1

Reisebüro-Automatisierung: Luftbilder abfragen und kategorisiert speichern

Modul	IET-122 – Praxisarbeit
Eingereicht von	Jonas Vetsch Stammgruppe LeutertHerbstVetsch
Eingereicht bei	Claude Fankhauser
Datum	24. November 2025

Änderungsverzeichnis

Datum	Version	Änderung	Autor
17.04.2023	0.5	Vorlage	Urs Dummermuth
24.11.2025	1	Separat geführte Dokumentation und handschriftliche Notizen in diese Dokumentationsvorlage hier übertragen. Reinschrift, Grammatikprüfung und stilistische Schlussüberarbeitung. Bereit zur Abgabe.	Jonas Vetsch

Inhaltsverzeichnis

1 Ziele und Anforderungen ... 3
1.1 Einleitung ... 3
1.2 Zweck des Skriptes ... 3
1.3 Ziele nach SMART ... 3
1.4 Sinn und Struktur der Text-Datei, die eingelesen wird ... 4
1.5 Anforderungen ... 4

2 Ablaufdiagramm ... 6
2.1 PAP ... 6
2.2 Kommentar / Beschreibung ... 7

3 Skript (Realisierung) ... 8
3.1 Technologie Python ... 8
3.2 Ein- und Ausgabe ... 8
3.3 Kontrollstrukturen ... 9

4 Integration und Sicherheit ... 10
4.1 Implementierung ... 10
4.2 Sicherheit ... 10
4.3 Kompatibilität ... 11
4.4 Betrieb und Wartung ... 11

5 Usecases und Testfälle ... 12
5.1 Usecase: Automatisiertes Herunterladen von Luftbildern ... 12
5.2 Testfälle ... 14

6 Reflexion ... 20
6.1 Journal Jonas Vetsch (20.11. – 24.11.2025) ... 20
6.2 Auswertung ... 22
6.3 Zielprüfung ... 22
6.4 Fazit ... 23

7 Anhang: Skriptlisting ... 24

1 Ziele und Anforderungen

1.1 Einleitung

Die Grundidee entstand im Gespräch zwischen Simon und Jonas aus unserer gemeinsamen Faszination für Karten und Luftbilder. Als wir die GeoAdmin-API ausprobierten, erkannten wir schnell, dass wir damit eine automatisierte Lösung mit Praxisnutzen entwickeln könnten. Daraus ergab sich die Idee für unser Skript, das Reisebüros die Arbeit erleichtert und wiederkehrende Aufgaben effizient übernimmt.

Ich selbst bringe bereits solide Vorkenntnisse in Python mit. Zudem habe ich Kenntnisse im Umgang mit APIs und beim Strukturieren kleinerer Softwareprojekte. Das aktuelle Vorhaben eignet sich hervorragend, um diese Grundlagen weiter zu vertiefen. Besonders reizt mich, eine API nicht nur zum Testen zu verwenden, sondern sie in eine klare, praxisnahe Funktionalität zu überführen.

Mit diesem Projekt möchte ich insbesondere das konsequente Anwenden guter Softwarearchitektur weiter trainieren. Ich möchte lernen, meine Programme stärker zu modularisieren: klare Trennung von Verantwortlichkeiten, saubere Dateistrukturen und eine Main.py, die nur noch die zentrale Logik enthält. Diese „Separation of Concerns“ systematisch umzusetzen ist ein zentrales Lernziel. Insgesamt möchte ich effizienter, strukturierter und professioneller programmieren.

1.2 Zweck des Skriptes

Mit dem Skript «Reisebüro-Automatisierung: Luftbilder abfragen und kategorisiert speichern» geben wir Reisebüros ein sinnvolles Tool an die Hand, um eine repetitive Aufgabe im Alltag zu automatisieren. Wenn ein Reisebüro eine Reise durch die Schweiz für einen Kunden plant, muss es für das Pitchdeck (PowerPoint-Präsentation, um die Reise zu verkaufen) und für die Detailplanung der Reise (nachdem die Reise verkauft wurde) immer mühsam Screenshots der wichtigsten Orte der Reise auf Google Maps machen. Damit ist nun Schluss.

Schluss mit mühsamen Screenshots und Snipping-Tool Einsätzen auf Google-Maps und zuschneiden auf Paint. Dank unserem Skript schreibt ein Mitarbeiter des Reisebüros die Adressen der gewünschten Orte einfach in eine Textdatei und kriegt innert Sekunden zu jedem Ort das Luftbild als JPEG-Datei. Und die Luftbilder sind zudem noch besser aufgelöst als bei Google Maps.

Der Nutzer gibt lediglich den Pfad des Textdokuments mit den Adressen der gewünschten Orte und den Pfad des gewünschten Zielordners für die Luftbilder ein. Danach liest das Skript die Adressen aus dem Textdokument und macht zu jeder Adresse einen API-Request an die GeoAdmin API. Von dort werden die Luftbilder heruntergeladen und direkt in den gewünschten Output-Ordner gespeichert. Am Schluss werden eine kurze Bestätigungsnachricht und allfällige Fehler ausgegeben.

1.3 Ziele nach SMART

1. Das Skript kann bis zur Abgabe der LB1.2, die vom Nutzer bereitgestellte Textdatei vollständig einlesen und Adresse und Unterordner korrekt trennen.
2. Bis zur Abgabe der LB1.2 liefert das Skript zu mindestens 90% der in der Textdatei aufgeführten Adressen erfolgreich ein Luftbild von der GeoAdmin API.

3. Das Skript speichert bis zur Abgabe der LB1.2 die erfolgreich heruntergeladenen Luftbilder automatisch in einen Unterordner mit korrektem Namen im Zielordner. Der Unterordner wird nur erstellt, sofern es ihn noch nicht gibt.

1.4 Sinn und Struktur der Text-Datei, die eingelesen wird

Die Textdatei, die dem Skript als Eingabe mitgegeben werden muss, ist vom Benutzer wie folgt anzulegen:

[Adresse des Ortes];[Name des Unterordners]
[Adresse des Ortes];[Name des Unterordners]
etc.

Pro Zeile darf nur eine Adresse stehen. Die Adresse muss in der Schweiz liegen. Sie muss durch ein Semikolon vom Namen des Unterordners abgetrennt werden. Es wird zuerst die Adresse und danach der Name des Unterordners eingeben. Damit werden die abgefragten Bilder automatisch in sinnvollen Unterordner abgelegt. So kann der Nutzer aus dem Reisebüro z.B. ein Unterordner für Hotels, ein Unterordner für Ausflugsziele und ein Unterordner für Bahnhöfe anlegen. Damit bleiben die Dateiausgaben unseres Skripts auch bei vielen Ausgaben stets übersichtlich.

Beispielinhalt der Textdatei

```
Bundesplatz 3, 3005 Bern;Ausflugsziel
Bahnhofplatz 10, 3011 Bern;Bahnhof
Bahnhofplatz 15, 8001 Zürich;Bahnhof
Heimplatz 1, 8001 Zürich;Kultur
Grossmünsterplatz 8, 8001 Zürich;Ausflugsziel
Quai Gustave-Ador 92, 1207 Genève;Ausflugsziel
Klosterhof 6D, 9000 St. Gallen;Kultur
Avenue de Chillon 21, 1820 Veytaux;Ausflugsziel
Kapellplatz 1, 6004 Luzern;Ausflugsziel
Rheinfallquai 30, 8212 Neuhausen am Rheinfall;Ausflugsziel
Kirchstrasse 8, 3920 Zermatt;Kultur
```

1.5 Anforderungen

1.5.1 Allgemeine Anforderungen

1. Der Benutzer (Mitarbeiter des Reisebüros, das Reisen nur innerhalb der Schweiz anbietet) kann mit einer selbst erstellten Textdatei automatisiert Luftbilder von Adressen abfragen.

1.5.2 Eingaben und Ausgaben

2. Das Skript fragt den Nutzer nach dem Pfad seiner Text-Datei.
3. Das Skript fragt den Nutzer nach dem Pfad bei dem der Ordner mit den abgefragten Bildern erstellt werden soll.
4. Am Schluss gibt das Skript eine Rückmeldung an den Nutzer in der erkennbar ist, dass das Skript nun abgeschlossen ist.

1.5.3 Programmtechnische Anforderungen

5. Das Skript prüft, ob die angegebene Textdatei existiert und lesbar ist. Falls nicht, wird eine verständliche Fehlermeldung ausgegeben und das Skript beendet sich sauber.
6. Das Skript liest alle Adressen (je auf einer neuen Zeile gespeichert) Zeile für Zeile ein.
7. Das Skript liest bei jedem Ordner auch den dazugehörigen Namen für den Unterordner ein.
8. Das Skript fragt zu jeder Adresse die Koordinaten und dazu dann das Luftbild bei der GeoAdmin API (https://api3.geo.admin.ch/) ab.
9. Das Skript legt das erhaltene Luftbild in den von Nutzer eingegebenen Ordner ab. Dabei erstellt das Skript automatisiert den korrekten Unterordner, sollte dieser nicht schon existieren. Das Bild erhält als Dateiname den Namen des Ortes, so wie er vom Nutzer im Textfile vorgegeben wurde. Der Unterordner erhält als Ordnername den Namen, wie er nach dem Semikolon im Textfile vorgegeben wurde.

2 Ablaufdiagramm

2.1 PAP

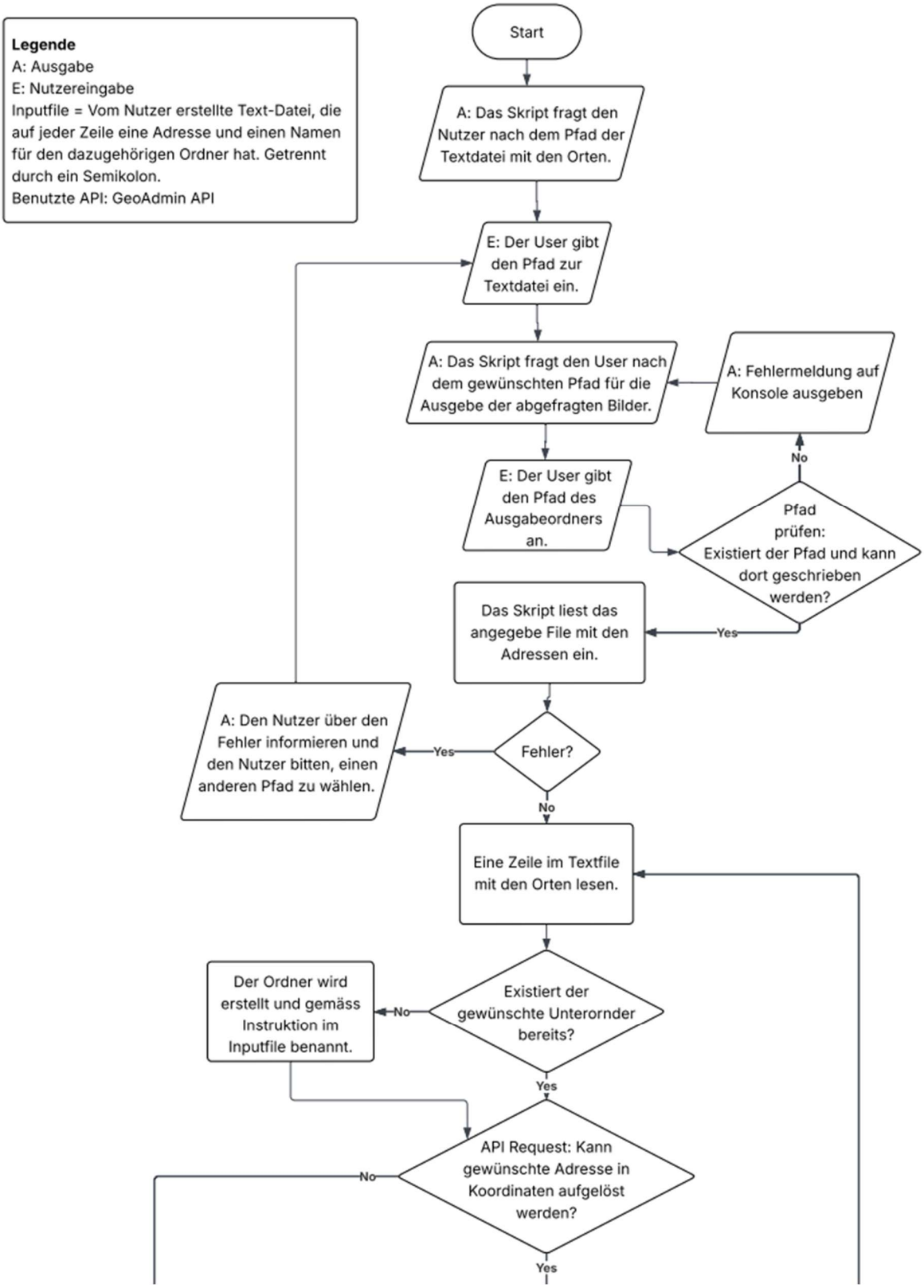


Abbildung 1 PAP Teil 1

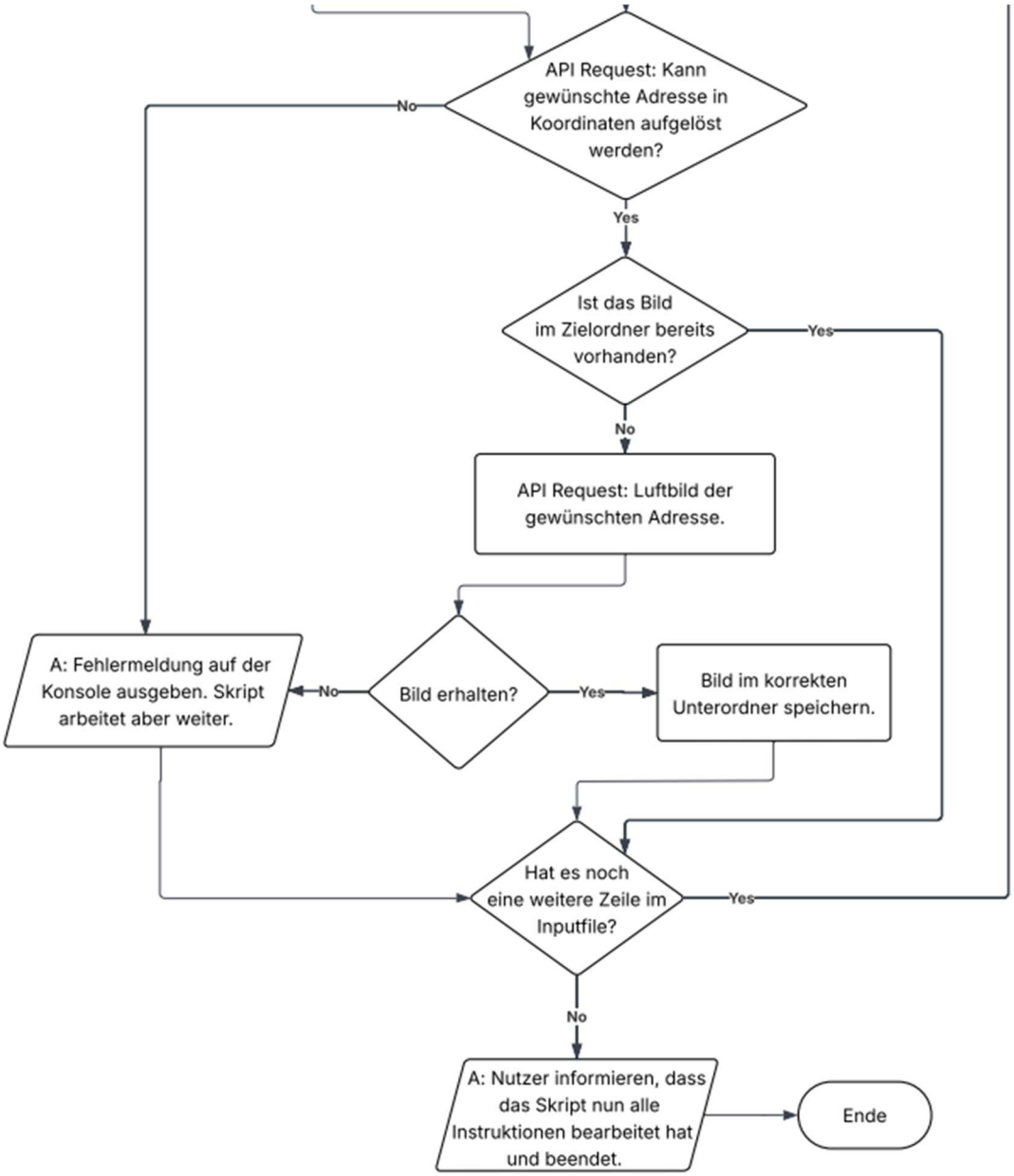


Abbildung 2 PAP Teil 2

2.2 Kommentar / Beschreibung

Der PAP beschreibt schrittweise, wie das Skript mit der GeoAdmin API arbeitet und stellt sicher, dass der Nutzer gut abgeholt wird. Zuerst fragt das Skript nach dem Pfad zur Textdatei mit den Orten und nach dem Ausgabeordner. Beide Pfade werden auf Existenz und Schreibrecht geprüft, bei Fehlern erhält der Nutzer eine klare Meldung.

Anschliessend wird die Inputdatei eingelesen. Treten dabei Probleme auf, informiert das Skript ebenfalls und bricht sauber ab. Danach werden die Zeilen nacheinander verarbeitet: Zu jedem Ort wird geprüft, ob der gewünschte Unterordner bereits existiert, sonst wird er erstellt.

Dann wird über einen API-Request die Adresse in Koordinaten übersetzt. Wenn das nicht funktioniert, wird eine Fehlermeldung ausgegeben, das Skript läuft aber weiter. Nur wenn das Bild im Zielordner noch nicht vorhanden ist, wird es heruntergeladen und im korrekten Unterordner gespeichert. Am Schluss informiert das Skript den Nutzer, dass alle Instruktionen abgearbeitet sind.

3 Skript (Realisierung)

3.1 Technologie Python

Für dieses Projekt habe ich mich bewusst für Python entschieden. Obwohl Bash oder PowerShell für einfache Automationen ausreichen würden, wäre die Arbeit mit APIs, Dateien und Fehlerbehandlung in diesen Umgebungen deutlich mühsamer geworden. Python bietet mir eine sehr klare Syntax, viele hilfreiche Bibliotheken und eine gute Lesbarkeit – gerade weil ich während der Lehre noch viel dazulerne, war das die angenehmste Wahl.

Das Skript besteht aus mehreren Modulen, die ich selbst strukturiert habe. Dabei war mir wichtig, dass jede Datei nur eine Aufgabe übernimmt. Die Datei **`main.py`** steuert den gesamten Ablauf des Programms. Die Logik rund um API-Anfragen und das Herunterladen der Luftbilder liegt in **`aerialLogic.py`**. Für alle Eingaben und Ausgaben des Nutzers habe ich die Datei **`userInteraction.py`** genutzt, die z. B. die Instruktionsdatei einliest oder den Ausgabeordner prüft. Kleine Hilfsfunktionen wie das Aufsplitten einer Zeile oder das Prüfen, ob ein Bild schon existiert, liegen in **`helperFunctions.py`**. Und für das Einhalten des API-Limits gibt es **`logs.py`**, das alle API-Requests loggt und das Tempo automatisch drosselt, wenn es nötig wird.

Diese Trennung hat mir sehr geholfen, nicht den Überblick zu verlieren. Am Anfang wirkte das Projekt noch komplex, aber sobald die Dateien klar strukturiert waren, fügte sich mit der Zeit alles sauber zusammen.

3.2 Ein- und Ausgabe

Die Ein- und Ausgaben habe ich bewusst einfach gehalten, damit das Skript ohne GUI funktioniert. Die wichtigste Eingabe ist die Datei input.txt, die pro Zeile eine Adresse und einen Kategorienamen enthält – getrennt durch ein Semikolon. Ein Beispiel aus meiner Testeingabe war etwa:

```
Bundesplatz 3, 3005 Bern;Ausflugsziel
```

Der Nutzer kann den Pfad zu dieser Datei frei eingeben. Falls nichts eingegeben wird, gehe ich davon aus, dass input.txt im selben Ordner wie das Skript liegt.

Als zweite Eingabe fragt das Skript nach dem Ausgabeordner. Auch hier ist die Standardoption **`./output`**. Ich prüfe automatisch, ob ich dort Dateien erstellen darf, indem ich eine kleine temporäre Datei anlege. Das ist wichtig, um Fehler durch fehlende Schreibrechte später im Skript zu vermeiden.

Die Ausgabe geschieht laufend über die Konsole. Ich informiere z. B., wenn ein Bild bereits existiert und der Download dadurch übersprungen wird, oder wenn GeoAdmin einen Ort nicht kennt.

Am Ende meldet das Skript, dass alle Zeilen verarbeitet wurden, bevor es die Logdatei bereinigt.

3.3 Kontrollstrukturen

Die wichtigste Schleife ist die Verarbeitung jeder Zeile der Instruktionsdatei. Ich gehe Zeile für Zeile durch, ignoriere Kommentare (Zeilen mit `#`) und rufe für jede gültige Zeile **`loadAerialImage()`** auf. Damit habe ich den gesamten Ablauf zentral gesteuert.

Innerhalb der Funktionen kommen dann mehrere Entscheidungen zum Einsatz:

- Existiert das Bild schon? Dann spare ich mir zwei API-Requests.
- Hat die GeoAdmin-API überhaupt Koordinaten gefunden? Wenn nicht, stoppe ich für diesen Ort.
- Ist der Zielordner vorhanden? Wenn nicht, lege ich ihn automatisch an.
- Wurden in der vergangenen Minute bereits die von GeoAdmin vorgesehenen 20 Requests pro Minute abgesetzt? Dann warte ich automatisch mithilfe meiner **`safeRequest()`** Methode.

Gerade diese automatische Limitierung der Requests war ein Lernmoment für mich: Ich musste recherchieren und verstehen, wie man Logs auswertet und wie man einen Prozess in einer while-Schleife pausiert, bis eine Bedingung wieder erfüllt ist.

4 Integration und Sicherheit

4.1 Implementierung

Bei der Implementierung war mir wichtig, dass das Skript nicht einfach nur „irgendwie“ funktioniert, sondern dass es sich wie ein gut durchdachtes Werkzeug anfühlt. Deshalb habe ich das Programm modular aufgebaut und die einzelnen Schritte sauber voneinander getrennt. Die Hauptlogik in **`main.py`** bildet das Rückgrat des Projekts. Hier rufe ich der Reihe nach die Benutzerinteraktion, die Verarbeitung der Instruktionsdatei und den Download der Bilder auf.

In **`aerialLogic.py`** liegt dann der Kern der inhaltlichen Arbeit: Die API-Anfragen, das Verarbeiten von Koordinaten, das Erstellen der Bounding-Box (das ist der Bereich auf der Karte, der im Luftbild zu sehen sein soll) und das Speichern der Bilder. Dank diesem Auslagern der Logik (ein klarer «separation of concerns») musste ich nicht in einer einzigen Datei arbeiten, sondern konnte mich immer auf das konzentrieren, was gerade relevant war.

Besonders spannend fand ich, wie alles zusammenspielt:

– Die User-Eingaben aus **`userInteraction.py`** steuern den Ablauf.

– Die Hilfsfunktionen aus **`helperFunctions.py`** unterstützen dabei, Fehler zu vermeiden (z. B. falsche Pfade oder doppelte Bilder).

– Und die Request-Steuerung aus **`logs.py`** sorgt dafür, dass das Skript nicht die API-Limite der GeoAdmin-Server überschreitet.

Das Zusammenspiel dieser Komponenten hat mir gezeigt, wie wertvoll es ist, eine Software nicht monolithisch, sondern in klaren Bausteinen zu denken. Bei Änderungswünschen – und die hatte ich einige während der Entwicklung – konnte ich vieles anpassen, ohne andere Teile zu beschädigen.

4.2 Sicherheit

Errorhandling ist nicht zu unterschätzen. Das habe ich in diesem Projekt wieder mal aus erster Hand bemerkt. Zudem ist Errorhandling meist detaillierter zu implementieren, als man es sich in der Konzeptphase vorstellt. So wurden mir während der Implementierung immer wieder neue Fehler bewusst, die entstehen können und im Errorhandling abgedeckt sein sollten.

Der erste Punkt betrifft die Ordner- und Pfadvalidierung. Sobald der Nutzer einen Ausgabepfad eingibt, prüfe ich nicht nur, ob dieser existiert, sondern auch, ob Python dort überhaupt schreiben darf. Dafür erstelle ich eine temporäre Datei. Klappt es nicht, wird dies sauber gemeldet und der Nutzer kann einen anderen Pfad auswählen.

Ein zweiter Aspekt betrifft die Fehlerbehandlung innerhalb der API-Logik. Jede API-Reaktion wird geprüft:

– War der Request erfolgreich?

– Hat die API überhaupt einen Ort gefunden?

– Habe ich ein Bild erhalten?

Letztlich wollte ich den (möglicherweise nicht technikaffinen Nutzer) vor ungewolltem API-Missbrauch schützen. Denn: Ich denke, dass man davon ausgehen muss, dass Resebüromitarbeiter*innen, die die primären Nutzer*innen des Skripts sein werden, nicht wissen, was eine API ist.

Die GeoAdmin-API hat ein Fair-Use-Limit von 20 Anfragen pro Minute. Würde ich zu schnell zu viele Orte abarbeiten, könnte die*der Reisebüromitarbeiter*in versehentlich eine Rate-Limit-Sperre auslösen – oder im schlimmsten Fall als „problematischer Client“ eingestuft werden. Darum protokolliere ich jede API-Anfrage in **`logs.txt`** mit einem Zeitstempel. Die Funktion **`safeRequest()`** prüft vor jedem API-Request, ob in der letzten Minute bereits 20 Requests gemacht wurden, und pausiert gegebenenfalls das Skript automatisch. So behalte ich jederzeit die Kontrolle über die Geschwindigkeit und riskiere keine Sperrung.

4.3 Kompatibilität

Einer der Vorteile von Python ist, dass mein Skript unabhängig vom Betriebssystem läuft. Ich habe es sowohl auf Windows wie auch auf macOS getestet. Ausser der Installation der requests-Bibliothek braucht das Skript keine weiteren Abhängigkeiten.

Da sämtliche API-Anfragen standardisierte HTTP-Requests sind und das Skript keinerlei GUI verwendet, gibt es keine Unterschiede in der Funktionalität zwischen den Systemen. Auch das Dateiformat **`.jpg`** ist universell kompatibel und kann auf jedem System geöffnet werden.

4.4 Betrieb und Wartung

Ich habe versucht, das Skript so zu gestalten, dass es langfristig stabil bleibt. Ein grosser Vorteil ist, dass die Logik komplett in Python geschrieben ist und keine externen Tools benötigt. Dadurch lassen sich Updates sehr leicht vornehmen.

Die Pflege beschränkt sich hauptsächlich auf zwei Bereiche:

1. Logdatei reinigen – Das Skript führt das am Ende automatisch durch, sodass die Datei nicht unendlich wächst.
2. API-Änderungen überwachen – Sollte GeoAdmin zukünftig Parameter ändern, müsste ich gegebenenfalls URLs, Parameter oder JSON-Strukturen anpassen. Da die Logik klar ausgelagert ist (separation of concerns), wäre das aber schnell erledigt.

Für den Betrieb genügt es, **`input.txt`** anzupassen und das Skript auszuführen. Die Struktur eignet sich gut dafür, irgendwann weitere Funktionen einzubauen – zum Beispiel andere Kartenlayer oder zusätzliche Metadaten. Die Basis steht stabil.

5 Usecases und Testfälle

5.1 Usecase: Automatisiertes Herunterladen von Luftbildern

Dieser Usecase beschreibt den typischen Ablauf, wie eine Person das Skript in der Praxis nutzen würde. So möchte z.B. eine Mitarbeiterin eines Reisebüros, eine Projektplanerin oder eine Geografiestudentin automatisch Luftbilder generieren.

Der Ablauf beginnt damit, dass der Nutzer eine Instruktionsdatei (**`input.txt`**) mit seinen Orten erstellt. Diese enthält pro Zeile die Adresse und einen gewünschten Unterordner, getrennt durch ein Semikolon, so zum Beispiel:

```
Bundesplatz 3, 3005 Bern;Ausflugsziel
Bahnhofplatz 15, 8001 Zürich;Bahnhof
```

Ich starte das Skript und bekomme zuerst eine kurze Einführung. Anschliessend werde ich gefragt, wo sich meine Instruktionsdatei befindet. Wenn ich einfach Enter drücke, sucht das Skript automatisch im gleichen Verzeichnis. Danach gebe ich an, wohin die Luftbilder gespeichert werden sollen.

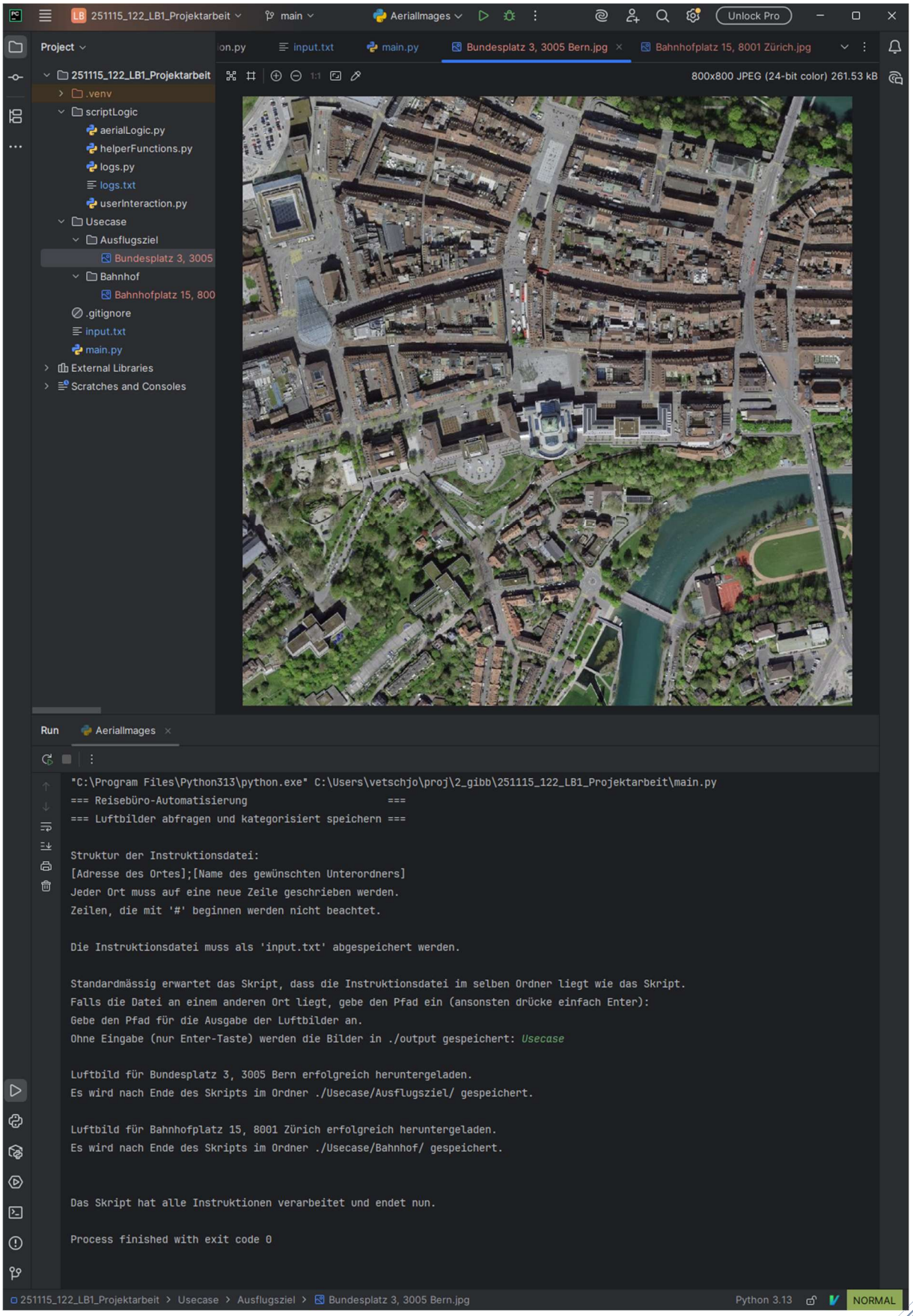
Sobald diese Vorbereitungen abgeschlossen sind, übernimmt das Skript alles Weitere automatisch:

- Es liest jede Zeile ein,
- findet die Koordinaten der gewünschten Adressen über die GeoAdmin-Search-API,
- lädt das Luftbild über die WMS-Schnittstelle,
- speichert die Bilder in der gewünschten Ordnerstruktur,
- überspringt Bilder, die bereits existieren
- und wartet automatisch, falls die API-Limite erreicht wurde

Am Schluss erhalte ich eine klare Abschlussmeldung, und die Logdatei wird automatisch bereinigt.

Dieser Usecase eignet sich für beliebig viele Orte und funktioniert auch bei falschen Eingaben oder unbekannten Orten zuverlässig, ohne abzustürzen.

Beweis der Durchführung Usecase



5.2 Testfälle

Ich habe drei Testfälle ausgewählt, die typische Situationen abdecken:

- Ein gut funktionierender Durchlauf
- Ein Fehler in der Instruktionsdatei
- Eine tatsächliche Fehlersituation über die API

Alle Testfälle wurden mit dem Skript durchgeführt.

5.2.1 Testfall 1 – Standardablauf

Ziel: Überprüfen, ob ein vollständiger, korrekter Ablauf funktioniert.

Vorbedingungen

- `input.txt` existiert und enthält gültige Adressen.
- Internetverbindung ist vorhanden.
- Ausgabeordner ist beschreibbar.

Eingabe (Beispielinhalt input.txt)

```
Bundesplatz 3, 3005 Bern;Ausflugsziel

Kapellplatz 1, 6004 Luzern;Ausflugsziel
```

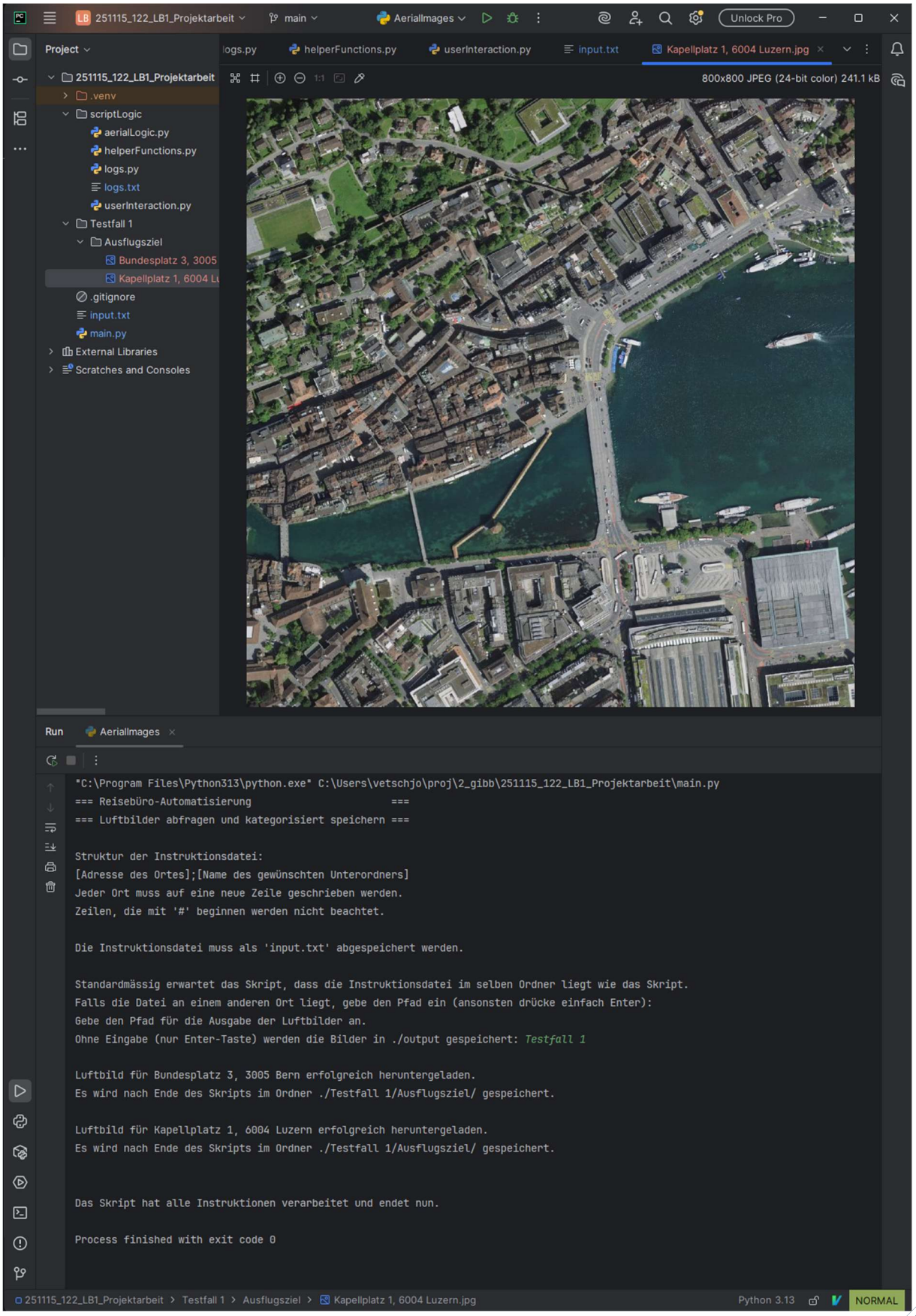
Erwartetes Verhalten

- Skript erkennt beide Zeilen.
- GeoAdmin findet beide Orte.
- Die Bilder werden erfolgreich heruntergeladen und in den Unterordnern gespeichert.
- Die Konsole zeigt Erfolgsmeldungen wie:
- „Luftbild für Bundesplatz 3, 3005 Bern erfolgreich heruntergeladen.“
- Skript endet ohne Fehler.

Ergebnis

Test erfolgreich. Der Ablauf entspricht der Erwartung.

Beweis der Durchführung Testfall 1



5.2.2 Testfall 2 – Falscher oder unvollständiger Pfad zur Instruktionsdatei

Ziel: Prüfen, ob das Skript sauber reagiert, wenn der Nutzer einen falschen Pfad zur Instruktionsdatei eingibt.

Durchführung
Beim Start gebe ich bewusst einen falschen Pfad ein, zum Beispiel:

```
C:/DiesenPfadGibtEsNicht/hierLiegtKeineInstruktionsdatei
```

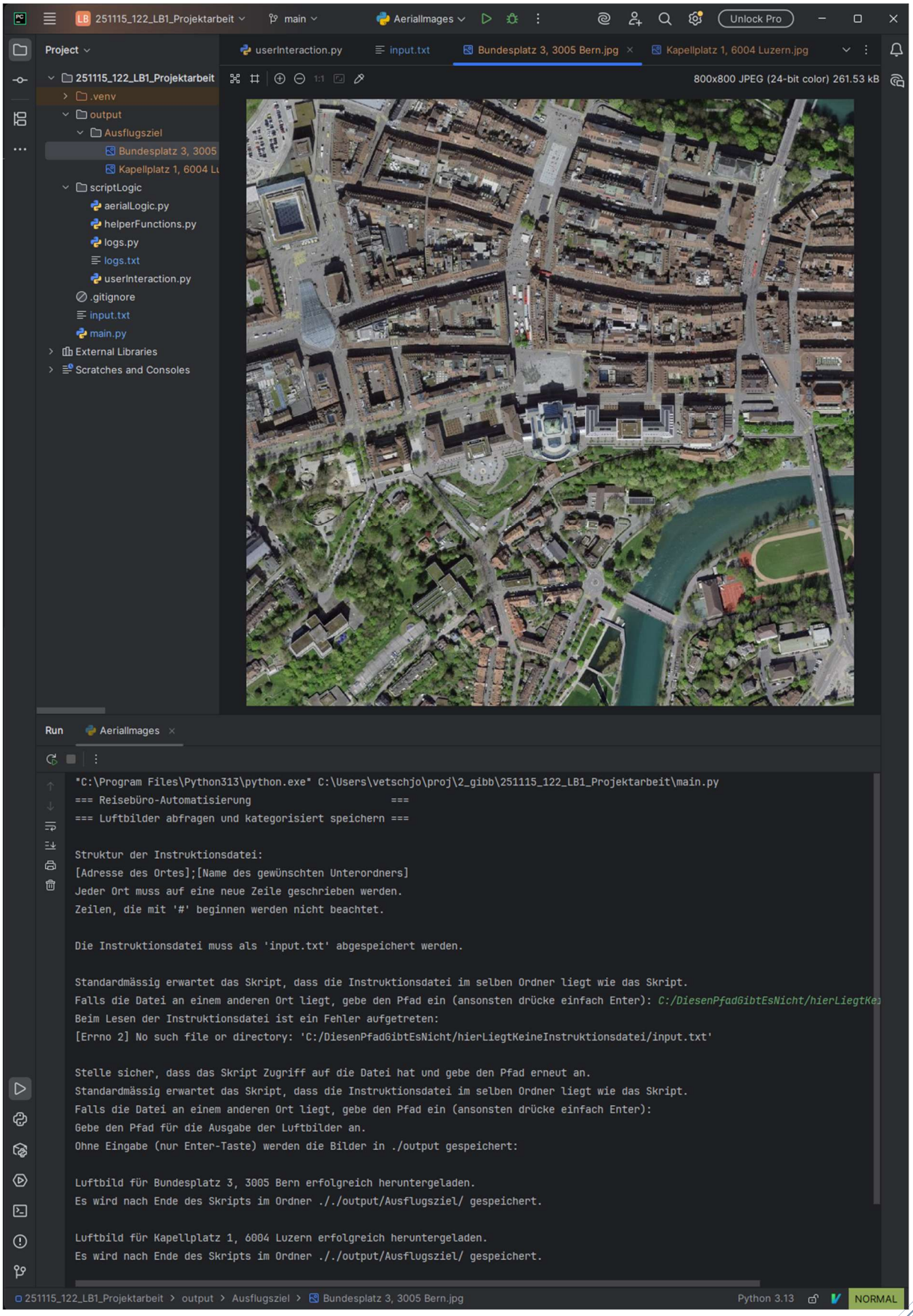
Erwartetes Verhalten

- Skript gibt eine klare Fehlermeldung aus:
- „Beim Lesen der Instruktionsdatei ist ein Fehler aufgetreten“
- Das Skript fordert mich erneut auf, einen gültigen Pfad einzugeben.
- Keine Abstürze, keine kryptischen Fehler.

Ergebnis

Test erfolgreich. Das Skript fängt Fehler sauber ab und erlaubt eine korrekte Neueingabe.

Beweis der Durchführung Testfall 2



5.2.3 Testfall 3 – Ungültige Adresse bzw. API findet den Ort nicht

Ziel: Überprüfen, wie das Skript reagiert, wenn die GeoAdmin-Suche nichts findet.

Eingabe (aus input.txt)

```
Teststrasse 123, 1234 Musterstadt;Testfall
```

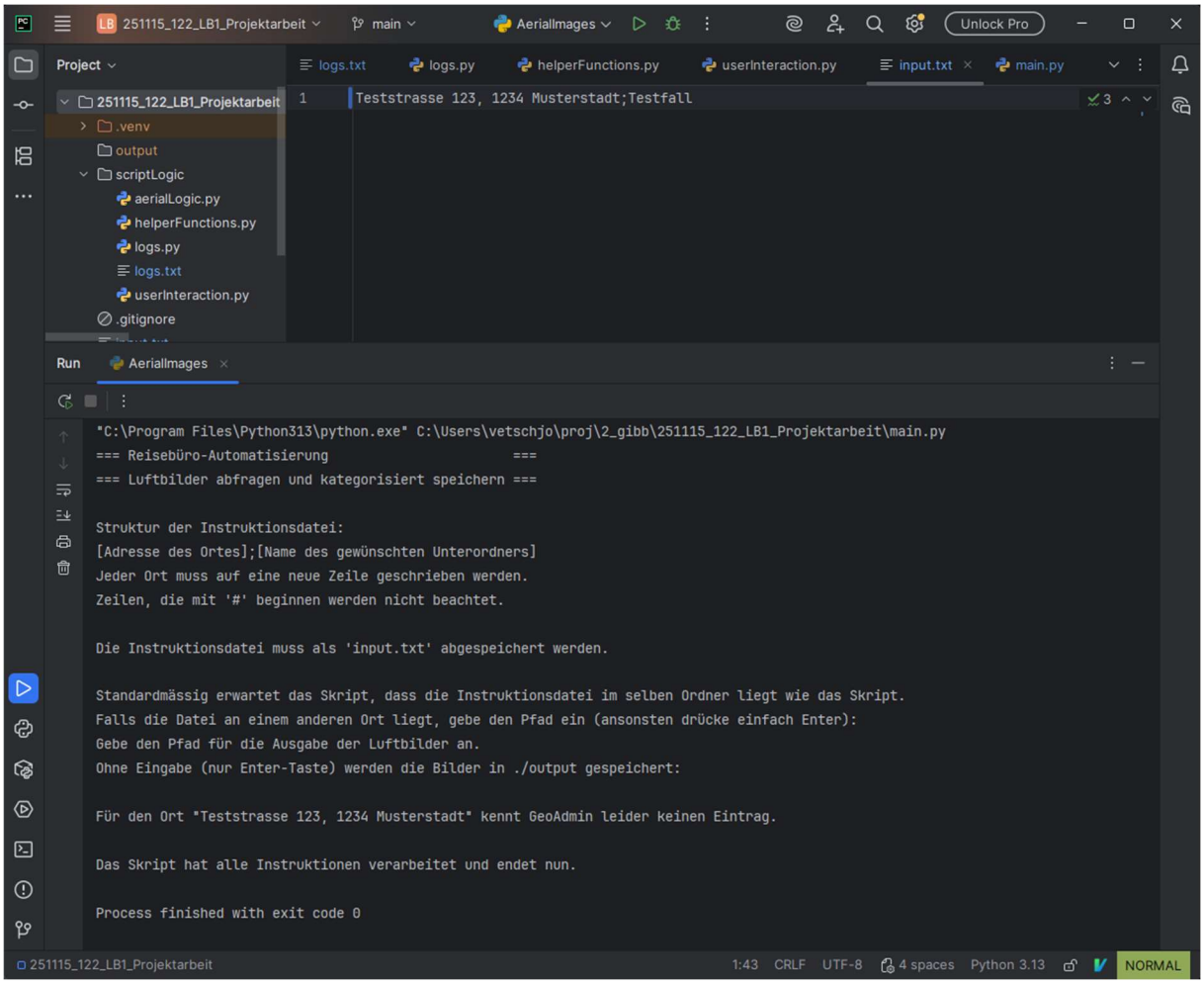
Erwartetes Verhalten

- Skript sendet API-Request, erhält aber keine „results“.
- Konsolenausgabe: „Für den Ort ‘Teststrasse 123, 1234 Musterstadt’ kennt GeoAdmin leider keinen Eintrag.“
- Der Ort wird nicht weiter verarbeitet.
- Kein Bild wird gespeichert.
- Das Skript fährt normal mit der nächsten Zeile fort.

Ergebnis

Test erfolgreich. Das Skript bricht nicht ab und zeigt eine verständliche Rückmeldung.

Beweis der Durchführung Testfall 3



6 Reflexion

6.1 Journal Jonas Vetsch (20.11. – 24.11.2025)

Tag	Tätigkeit	Bemerkungen
1	Projektstart und Grundstruktur	Heute habe ich das Projekt gestartet und mir zuerst Zeit genommen, das Grundgerüst mit mehreren Python-Modulen zu planen (main, userInteraction, aerialLogic, helperFunctions) und mir überlegt, welche Aufgaben in welche Datei gehören. Es war ein gutes Gefühl zu sehen, wie aus einer groben Idee eine erste Struktur entsteht. Nächster Schritt: Die GeoAdmin-API anbinden und erste Testanfragen machen.
2	Debugging der API und zwei unterschiedliche Schweizer Koordinatensysteme	Heute bin ich lange an einem Problem mit „falschen“ Bildern gesessen. Zuerst waren die Luftbilder komplett schwarz, später nur noch weiss. Die API hat keine klaren Fehlermeldungen geliefert aus ihrer Sicht war alles ok. Nach viel Ausprobieren habe ich gemerkt, dass ich zwei verschiedene Schweizer Koordinatensysteme durcheinandergebracht habe. Beim Geocoding kam das eine System zurück, beim GetMap-Request hatte ich aber das andere konfiguriert. Dadurch lag mein Ausschnitt wohl irgendwo ausserhalb der Schweiz. Nachdem ich das Koordinatensystem vereinheitlicht hatte, kamen endlich sinnvolle Luftbilder – ein grosser Aha-Moment.
3	Errorhandling und Pfadprüfungen	Heute habe ich mich intensiv mit Fehlerbehandlung beschäftigt: Prüfen, ob das Inputfile existiert, ob ich im Ausgabeordner schreiben darf und ob die Unterordner schon vorhanden sind. Ich musste zuerst recherchieren, wie man das in Python elegant löst, und habe einiges über `try/except`, `os.makedirs` und den Umgang mit Pfaden gelernt. Herausfordernd war vor allem der Unterschied zwischen Windows- und UNIX-Pfaden. Mit etwas Geduld und Internetsuche hat es dann aber funktioniert. Ich bin stolz, dass das Skript nun nicht mehr einfach abstürzt, sondern verständliche Fehlermeldungen ausgibt.
4	Logging, API-Limit und List-Comprehension	Heute habe ich zwei Dinge umgesetzt, die mir fachlich viel gebracht haben. Mir ist erst heute die Idee gekommen, dass ich ein Logging einbauen könnte. Ich habe mir während der bisherigen Entwicklung schon ein paar

		Mal die Frage gestellt, ob ich irgendwie sicherstellen könnte, dass das Ratelimit der API eingehalten wird. So habe ich heute zum ersten Mal ein Logging gebaut. Ich speichere bei jedem API-Request einen Timestamp. Vor jedem neuen Request prüfe ich, wie viele Anfragen in der letzten Minute gemacht wurden. Wenn es mehr als 20 sind, wartet das Skript automatisch. So kann theoretisch auch eine riesige Menge von z. B. 10'000 Luftbildern abgearbeitet werden, ohne das Fair-Use-Limit zu verletzen. Zweitens habe ich endlich List-Comprehensions verstanden und im Log-Cleanup eingesetzt. Früher wirkten diese Einzeiler wie Magie, jetzt sehe ich, wie elegant sie sind. Die Schlüsselerkenntnis hierbei war: Das erste Wort nach er '[' steht für das Element, das in diesem Durchgang aus der Liste genommen wird und in die neue Liste gespeichert wird. Es muss daher denselben Variablennamen haben wir das zweite Element, das genannt wird. Doch warum wird es nochmal erwähnt? Weil man damit noch Operationen auf dieses Element ausführen kann, z.B. `.upper()` oder `*2` oder so... Zum Abschluss habe ich den Code noch etwas aufgeräumt, die Kommentare verbessert und den Code auf GitHub für mich gespeichert. Damit bin ich ready für die Abgabe.

6.2 Auswertung

Wenn ich auf das Projekt zurückblicke, merke ich, wie viel ich unterwegs gelernt habe. Am Anfang stand nur die Idee, aus einer einfachen Textdatei mit Adressen automatisch Luftbilder zu erzeugen. Daraus ist ein kleines, aber recht «erwachsenes» Werkzeug geworden. Wie immer ist die Realisierung auch einiges umfangreicher, als man es sich zunächst vorgestellt hat. Doch ich verbringe sehr gerne Zeit mit Code, daher hat es mir nichts ausgemacht hier ein paar Extrastunden zu investieren.

Besonders wichtig in diesem Projekt war für mich die Strukturierung in mehrere Module. Durch die Aufteilung in `main`, `aerialLogic`, `userInteraction`, `helperFunctions` und `logs` konnte ich Probleme viel gezielter angehen. Das hat mir gezeigt, wie stark gute Struktur die Komplexität reduziert.

Die Arbeit mit der GeoAdmin-API war anspruchsvoller als gedacht. Schwarze und weisse Bilder ohne Fehlermeldung haben mir gezeigt, dass eine API zwar “korrekt” antworten kann, das Resultat aber trotzdem unbrauchbar ist. Erst als ich verstanden habe, wie die unterschiedlichen Schweizer Koordinatensysteme funktionieren und alles auf ein System umgezogen habe, kamen sinnvolle Bilder zum Vorschein. Das war ein entscheidender Lernschritt.

Ein weiterer Schwerpunkt war das Errorhandling: Pfade prüfen, fehlende Rechte sauber melden, mit ungültigen Orten umgehen. Ein gutes Skript muss nicht nur im Idealfall funktionieren, sondern vor allem dann, wenn etwas schiefgeht. Die Tests haben mir geholfen, das Verhalten robuster zu machen.

Sehr lehrreich war auch das Logging für das API-Limit. Ich habe das erste Mal ein eigenes System gebaut, das Requests mitzählt und automatisch wartet, wenn das Limit erreicht ist. Damit habe ich besser verstanden, wie man fair und kontrolliert mit externen Diensten umgeht.

Insgesamt habe ich sowohl fachlich (APIs, Koordinaten, Logging, List-Comprehension) als auch methodisch (strukturieren, testen, iterativ verbessern) viel mitgenommen.

6.3 Zielprüfung

6.3.1 Einlesen der Textdatei / Trennen von Adresse und Unterordner

Dieses Ziel sehe ich als erreicht: Die Textdatei wird vollständig eingelesen, leere und kommentierte Zeilen werden sinnvoll behandelt, und Adresse sowie Unterordner werden sauber am Semikolon getrennt.

6.3.2 Mindestens 90 % erfolgreiche Luftbilder

Unter meinen Tests mit realistischen, korrekten Adressen wurde dieses Ziel erreicht. Technisch ist das Skript so gebaut, dass alle gültigen Adressen ein Luftbild liefern, sofern die GeoAdmin-API einen Treffer findet und erreichbar ist. Bei Fantasieadressen oder Tippfehlern sind Fehlschläge erwartbar, werden aber korrekt gemeldet.

6.3.3 Automatisches Speichern im korrekten Unterordner

Auch dieses Ziel ist erreicht: Das Skript erstellt Unterordner nur dann, wenn sie noch nicht existieren, und speichert die Bilder mit passendem Namen dort ab. Bereits vorhandene Bilder werden erkannt und nicht doppelt heruntergeladen.

6.4 Fazit

Für mich war dieses Projekt mehr als nur eine Pflichtaufgabe. Ich habe ein Skript gebaut, das einen echten Nutzen hat, sauber strukturiert ist und auch mit Fehlern umgehen kann.

Ich nehme vor allem drei Kernpunkte mit:

- Struktur lohnt sich: Ein modularer Aufbau macht das Arbeiten angenehmer und spätere Anpassungen viel einfacher.
- Fehler gehören dazu: Gutes Programmieren heisst nicht, dass nie etwas schiefgeht, sondern dass das Programm mit Problemen sinnvoll umgehen kann.
- Lernen im Tun: Viele Dinge, die zuerst kompliziert wirkten (Koordinatensysteme, Logging, List-Comprehension), sind durch das konkrete Projekt verständlich geworden. Und wie so oft: Wenn man es einmal verstanden hat ist es gar nicht mehr so kompliziert.

Ich bin mit dem Endresultat sehr zufrieden: Das Skript erfüllt den Zweck, ist nachvollziehbar aufgebaut und kann bei Bedarf erweitert werden. Vor allem habe ich das Gefühl, einen echten Schritt Richtung „richtige“ Softwareentwicklung gemacht zu haben.

7 Anhang: Skriptlisting

7.1.1 main.py

```
import os
from scriptLogic.userInteraction import initialExplanationOfScript, get-
InstructionsFromFile, getOutputPath, scriptEnd
from scriptLogic.aerialLogic import loadAerialImage
from scriptLogic.logs import cleanUpLogs

# Disply basic explanation about script to user
initialExplanationOfScript()

# Get instructions from user
instructionsList = getInstructionsFromFile()

# Get path to output folder for aerial images
outputFolder = getOutputPath()

# Create output folder if it doesn't exist yet
os.makedirs(f'./{outputFolder}', exist_ok=True)

# Get and save aerial image for each instruction
for instruction in instructionsList:
    if not instruction.startswith('#'): # Ignore commented lines
        loadAerialImage(instruction, outputFolder)

# Inform user about end of script
scriptEnd()

# Only keep logs from the current day
cleanUpLogs()
```

7.1.2 input.txt

```
Bundesplatz 3, 3005 Bern;Ausflugsziel
Bahnhofplatz 10, 3011 Bern;Bahnhof
Bahnhofplatz 15, 8001 Zürich;Bahnhof
Heimplatz 1, 8001 Zürich;Kultur
Grossmünsterplatz 8, 8001 Zürich;Ausflugsziel
Quai Gustave-Ador 92, 1207 Genève;Ausflugsziel
Klosterhof 6D, 9000 St. Gallen;Kultur
Avenue de Chillon 21, 1820 Veytaux;Ausflugsziel
Kapellplatz 1, 6004 Luzern;Ausflugsziel
Rheinfallquai 30, 8212 Neuhausen am Rheinfall;Ausflugsziel
Kirchstrasse 8, 3920 Zermatt;Kultur
```

7.1.3 userInteraction.py

```
from scriptLogic.helperFunctions import formatFilePath
import os
import tempfile

# Display basic explanation about script
def initialExplanationOfScript():
    print("=== Reisebüro-Automatisierung                        ===")
    print("=== Luftbilder abfragen und kategorisiert speichern ===")
    print()

    print("Struktur der Instruktionsdatei:")
    print("[Adresse des Ortes];[Name des gewünschten Unterordners]")
    print("Jeder Ort muss auf eine neue Zeile geschrieben werden.")
    print("Zeilen, die mit '#' beginnen werden nicht beachtet.")
    print()
    print("Die Instruktionsdatei muss als 'input.txt' abgespeichert wer-
den.")
    print()


# Ask user to specify path to instruction file (text file with places)
def getInstructionsFromFile():
    approved = False
    while not approved:
        # Get instruction file path from user
        print("Standardmässig erwartet das Skript, dass die Instrukti-
onsdatei im selben Ordner liegt wie das Skript.")
        pathFromUser = input("Falls die Datei an einem anderen Ort
liegt, gebe den Pfad ein (ansonsten drücke einfach Enter): ")
        path = formatFilePath(pathFromUser, '.')

        # Read text file with instructions
        try:
            with open(f"{path}/input.txt", "r", encoding="utf-8") as in-
structionFile:
                instructionsRaw = instructionFile.read()

        # Unsuccessful file access
        except Exception as exception:
            print("Beim Lesen der Instruktionsdatei ist ein Fehler auf-
getreten:")
            print(exception)
            print()
            print("Stelle sicher, dass das Skript Zugriff auf die Datei
hat und gebe den Pfad erneut an.")

        # Successful file access
        else:
```

```
            approved = True
            return instructionsRaw.splitlines()


# Ask user to specify path for output of script results
def getOutputPath():
    approved = False
    while not approved:
        print("Gebe den Pfad für die Ausgabe der Luftbilder an. ")
        pathFromUser = input("Ohne Eingabe (nur Enter-Taste) werden die
Bilder in ./output gespeichert: ")
        path = formatFilePath(pathFromUser, './output')

        # Validate output folder
        try:
            # Check if output folder exists, else create it
            os.makedirs(path, exist_ok=True)

            # Create temporary file to check write permissions in output
folder
            testfile = tempfile.TemporaryFile(dir=path)
            testfile.close()

        # Invalid output folder
        except Exception as exception:
            print("Beim Versuch auf den angegebenen Ausgabeordner
zuzugreifen gabe es einen Fehler.")
            print(exception)
            print()
            print("Bitte versuche es nochmal.")

        # Valid output folder
        else:
            approved = True
            print() # Empty line for better visual separation on console
            return path


# Script end text
def scriptEnd():
    print() # Empty line for better visual separation on console
    print('Das Skript hat alle Instruktionen verarbeitet und endet
nun.')
```

7.1.4 logs.txt

```
# NOTE: Do not change the contents or name of this file,
# unless you know what you're doing.
```

```
# File keeps track of API requests so that the script can
# slow itself down to make sure it doesn't go over the
# fair use limit of 20 requests per minute.


2025-11-23 16:17:37.354015
```

7.1.5 logs.py

```
import time
from datetime import datetime, timedelta

logsInstructions = '''# NOTE: Do not change the contents or name of this
file,
# unless you know what you're doing.

# File keeps track of API requests so that the script can
# slow itself down to make sure it doesn't go over the
# fair use limit of 20 requests per minute.


'''



# Log the timestamp of a request to logfile
def logRequest():
    with open("./scriptLogic/logs.txt", "a", encoding="utf-8") as log-
File:
        timestamp = datetime.now()
        logFile.write(f'\n{timestamp}')


def getLogs():
    # Read the logs
    with open("./scriptLogic/logs.txt", "r", encoding="utf-8") as log-
File:
        logData = logFile.read()

    cleanLogs = []
    logsRaw = logData.splitlines()

    # [] Remove the instruction text
    for line in logsRaw[8:]:
        line = line.strip()
        if line == "":
            continue
        try:
            datetime.fromisoformat(line)
            cleanLogs.append(line)
        except:
```

```
            continue
    return cleanLogs


# Read logs and check if more than 20 requests have been made in the
past minute
def requestAllowed():
    # API Rate limit in requests per Minute
    requestsPerMinuteAllowed = 20

    logs = getLogs()

    oneMinuteAgo = datetime.now() - timedelta(minutes=1)
    pastMinute = 0

    for log in logs:
        dateTimeObject = datetime.fromisoformat(log)
        if dateTimeObject >= oneMinuteAgo:
            pastMinute += 1

    if pastMinute >= requestsPerMinuteAllowed:
        return False
    else:
        return True


# Stops execution until an API call can be safely made without
# going over the api limit
def safeRequest():
    while not requestAllowed():
        print("Das Skript wartet, um den API-Dienst nicht zu überlas-
ten...")
        time.sleep(3)


def cleanUpLogs():
    logs = getLogs()
    oneDayAgo = datetime.now() - timedelta(days=1)

    recentLogs = [log for log in logs if datetime.fromisoformat(log) >=
oneDayAgo]

    with open("./scriptLogic/logs.txt", "w", encoding="utf-8") as log-
File:
        logFile.write(logsInstructions)

        for log in recentLogs:
            print(log, file=logFile)
```

7.1.6 helperFunctions.py

```
import os

# Format file path to not end on '/', for empty path return a default
path
def formatFilePath(path, defaultPath):
    match path:
        case '':
            return defaultPath
        case p if p.endswith('/'):
            return p[:-1]
        case _:
            return path

# Split one line of user instruction by the ';'
def readUserInstruction(instruction):
    # Split instruction at ';'
    instructionAsList = instruction.split(";")

    return instructionAsList[0], instructionAsList[1]

# Check wheter given file exists at given path
def checkImageExisting(path, locationName):
    # First we check if the subfolder exists
    if not os.path.isdir(path):
        return False

    # If the subfolder exists, we check for an
    # existing image for the requested location
    files = os.listdir(path)
    imageExisting = False

    for file in files:
        file = file.replace('.jpg', '')
        if file == locationName:
            imageExisting = True

    if imageExisting:
        print(f'Im Ordner {path} wurde bereits ein Bild für {location-
Name} gefunden.')
        print('Das Skript verzichtet auf einen erneuten Download.')

    return imageExistin
```

7.1.7 aerialLogic.py

```
import requests
import re
import os
```

```
from scriptLogic.helperFunctions import readUserInstruction,
checkImageExisting
from scriptLogic.logs import logRequest, safeRequest


# Process one line of instruction: split instruction, get coordinates,
get and save areial image
def loadAerialImage(instruction, outputFolder):

    # Split user instruction into location and folder name
    locationFromInput, subfolder = readUserInstruction(instruction)

    # If requested aerial image is already in output folder: save 2 api
calls
    if checkImageExisting(os.path.join(outputFolder, subfolder), loca-
tionFromInput):
        return

    # Get coordinates and full location name from API
    x, y, locationFromAPI = requestGeoInformation(locationFromInput)

    # if successful, get aerial image from API and save to folder
    if x != '':
        requestAndSaveImage(x, y, locationFromInput, outputFolder, sub-
folder, zoomLevel = 750)


# Get coordinates and official name of a given location
def requestGeoInformation(locationFromUser):
    # Check logs to stay within API limits and wait if necessary
    safeRequest()

    # URL of geocoding API endpoint of GeoAdmin
    url = "https://api3.geo.admin.ch/rest/services/api/SearchServer"

    # Parameters for the API Request
    params = {
        'searchText': locationFromUser,  # Die Adresse, die geocodiert
werden soll
        'type': 'locations',  # Wir suchen nach geokodierten Orten
        'limit': 1,  # Maximale Anzahl der Ergebnisse
        'returnGeometry': 'true',  # Wir wollen die Geometrie (Koordi-
naten) im Ergebnis
        'sr': '2056'  # Neue Landesvermessung (2...../1....)
    }

    # Send request
    response = requests.get(url, params=params)
    logRequest()
```

```
    # Case: Got valid answer
    if response.status_code == 200:
        data = response.json()

        # Check if API found the requested location
        if data['results']:
            # Coordinates, in the swiss coordinate system 'Neue
Landesvermessung'
            x = float(data['results'][0]['attrs']['x'])
            y = float(data['results'][0]['attrs']['y'])

            # Official name of location
            locationNameRaw = data['results'][0]['attrs']['label']
            locationName = re.sub(r"<.*?>", "", locationNameRaw)
            # print(f'Für den Ort "{locationFromUser}" wurde "{location-
Name}" gefunden.')

            return x, y, locationName
        else:
            print(f'Für den Ort "{locationFromUser}" kennt GeoAdmin lei-
der keinen Eintrag.')
            return '', '', ''
    else:
        print(f"Für den Ort {locationFromUser} ist folgender Fehler
aufgetreten:")
        print(f"{response.status_code} - {response.text}")
        return '', '', ''


# Get aerial image for coordinates and save to given output folder with
given file name
def requestAndSaveImage(x, y, fileName, outputFolder, subfolder, zoom-
Level):
    # Check logs to stay within API limits and wait if necessary
    safeRequest()

    # Calculate image frame with desired zoom level
    minx = x - zoomLevel / 2
    miny = y - zoomLevel / 2
    maxx = x + zoomLevel / 2
    maxy = y + zoomLevel / 2

    # WMS 1.3.0 with EPSG:2056 requires this weired format: (y,x,y,x)
    bbox = f"{miny},{minx},{maxy},{maxx}"

    # Web Map Service API of Geo Admin
    url = "https://wms.geo.admin.ch/?"
```

```
    # Parameters for the API Request
    params = {
        "SERVICE": "WMS",
        "VERSION": "1.3.0",
        "REQUEST": "GetMap",
        "LAYERS": "ch.swisstopo.swissimage",
        "STYLES": "",
        "FORMAT": "image/jpeg",
        "TRANSPARENT": "FALSE",
        "CRS": "EPSG:2056",
        "BBOX": f"{bbox}",
        "WIDTH": "800",
        "HEIGHT": "800"
    }

    # Send request
    response = requests.get(url, params=params)
    logRequest()

    # Case: Got results
    if response.status_code == 200:
        # Create output sub folder in case it doesn't exist yet
        os.makedirs(f'./{outputFolder}/{subfolder}', exist_ok=True)

        # Save image
        with open(f"./{outputFolder}/{subfolder}/{fileName}.jpg", "wb")
as f:
            f.write(response.content)
        print(f"Luftbild für {fileName} erfolgreich heruntergeladen.")
        print(f"Es wird nach Ende des Skripts im Ordner ./{outputFol-
der}/{subfolder}/ gespeichert.")
        print()

    # Case: Didn't get results
    else:
        print(f"Die Luftbildanfrage für {fileName} gab folgenden Fehler:
{response.status_code} - {response.text}")
```